

Deepcache

00 Abstract

Diffusion model : 뛰어난 생성 능력, 막대한 계산 비용

⇒ 모델 구조 측면에서 확산 모델 가속화.

⇒ **DeepCache**

- 노이즈 제거 단계의 시간적 중복 활용
- 노이즈 제거 단계의 특징 재사용
- U-Net 에서, 고수준 특징을 저수준 단계에서 적은 비용으로 업데이트

pruning, distillation 등 재학습을 요구하는 방법보다 우수

현재 샘플링 기술과 호환 가능

DDIM, PLMS와 비교했을 때 동등/다소 개선 결과 보임

01 Introduction

Diffusion model

- 생성 능력 - image, text, audio, video → image editing, image super-enhancing , image-to-image translation , text-to-image generation, text-to-3D generation
- 하지만 느린 속도 - reverse phase의 denoising 과정에서 발생

⇒ 가속화

- 샘플링의 단계 수 줄이기
- inference overhead 줄이기 : pruning, distillation, quantization

기존의 방식은 모델의 크기를 압축하는 것에 집중했지만, 이를 재학습하기 위해 대규모 데이터가 필요함.

이 과정의 비용을 줄이기 위한 방법이 있지만, 재학습이 반드시 필요하고 경량화 과정 또한 비용을 요구하기 때문에 실용적이지 않다.

How to significantly reduce the computational overhead at each denoising step without additional training, thereby achieving a cost-free compression of Diffusion Models?

(추가학습 없이 denoising의 오버헤드를 줄여 적은 비용으로 모델을 압축할 수 있는 방법)

⇒ 노이즈 제거 과정에서 a significant temporal consistency in the high-level features between consecutive steps 라는 특징에 집중

⇒ 노이즈 제거 과정에서 연속적인 단계에서 얻는 특징들이 공통점을 갖는다는 점에 집중

⇒ 캐시 가능

Contributions

- DeepCache : 실행 중 모델 동적 압축 → 생성 속도 향상
- high-level features 간의 시간적 일관성 활용 → 중복 계산 방지, 1:N 전략
- CIFAR, LSUN-Bedroom/Churches, ImageNet, COCO2017, PartiPrompt 등의 데이터셋으로 검증
- DDPM, LDM, Stable Diffusion 테스트
- pruning, distillation 보다 좋은 성능

02 Related Work

High-dimensional image generation : GANs & VAEs - scalability issues(확장성 문제) → diffusion Probabilistic model - 느린 추론 속도

Optimized Sampling Efficiency

- DDIM : Neural ODEs(신경 미분 방정식) & non-Markovian process 연구
- Consistency Model : 하나의 모델 평가로 랜덤 노이즈 변환

- DSNO, Para DiGMS : 병렬 샘플링 기법, Fourier neural operators & Picars iter 사용하여 병렬 디코딩

Optimized Structural Efficiency

- Diff-pruning : 구조적인 가지치기
- SnapFusion : 효율적 구조 진화
- Spectral Diffusion : 주파수, 우선 정보 → 아키텍처 디자인 향상
- 양자화

03 Methodology

3.1 Preliminary

Forward and Reverse Process.

- 무작위 노이즈로부터 목표하는 분포와 유사한 샘플로 변환하는 것
 - 순방향 : 가우시안 노이즈가 T 단계동안 점진적으로 추가됨.
 - 베타는 노이즈 조정

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

- 역방향 : 노이즈 제거. $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ 부터 시작하여 목표하는 분포로 변환하는 것
- $\mathbf{x}_0$ 에 가까운 데이터 포인트로 변환

$$x_{t-1} \sim p_{\theta}(x_{t-1} | x_t) = \mathcal{N}\left(x_{t-1}; \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right), \beta_t \mathbf{I}\right)$$

High-level and Low-level Features in U-Net.

- 본래 의학 이미지 분할 위해 소개됨
- 저수준과 고수준 피쳐의 결합 능력 → skip connection
 - main branch, skip connection은 concatenation으로 합쳐
 - 다운스트림에서 업스트림으로 저수준의 정보 전달

$$\text{Concat}(D_i(\cdot), U_{i+1}(\cdot))$$

3.2. Feature Redundancy in Sequential Denoising

확산모델에서 denoising 과정의 순차성 → bottleneck(병목 현상) 발생 가능

이전에는 병목 현상 해결 위해 특정 단계 건너뛰는 전략 사용

→ 더 효율적인 추론 과정 만들기 위한 속성 찾기

Observation.

- 연속적인 단계에서, 고수준 특징이 시간적 유사성을 보임
 - 노이즈 제거 과정의 인접 단계 사이 시간적 유사성 → 이 과정에서 변화가 작음
 -

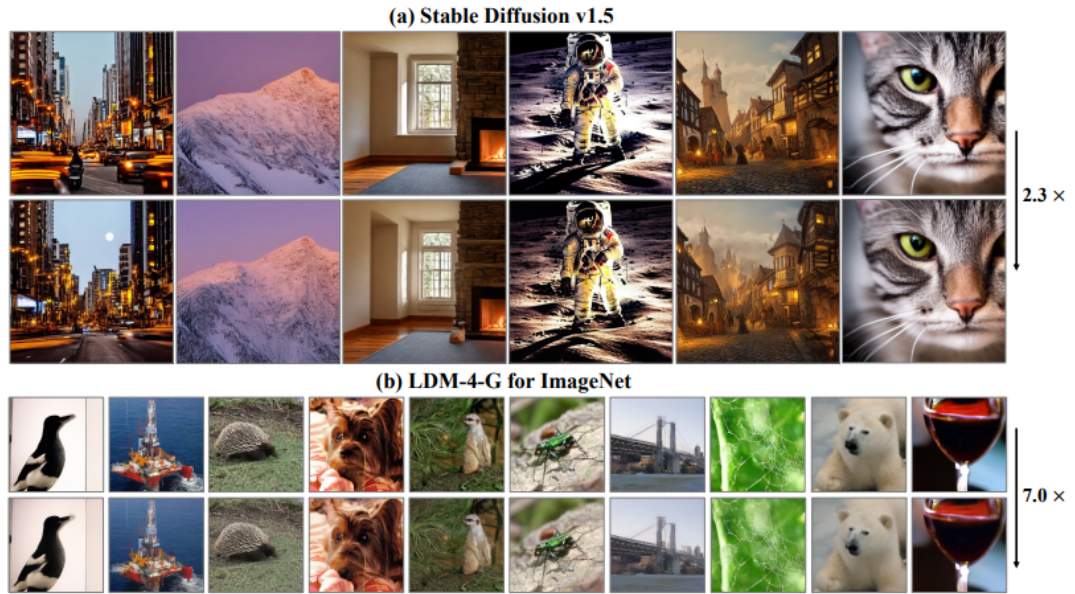


Figure 1. Accelerating Stable Diffusion V1.5 and LDM-4-G by $2.3\times$ and $7.0\times$, with 50 PLMS steps and 250 DDIM steps respectively.

- 10% 인접 단계에서 0.95 이상의 유사성 → 점진적 변화 일어남, Stable Diffusion, LDM, DDPM 에서 관찰 가능

○

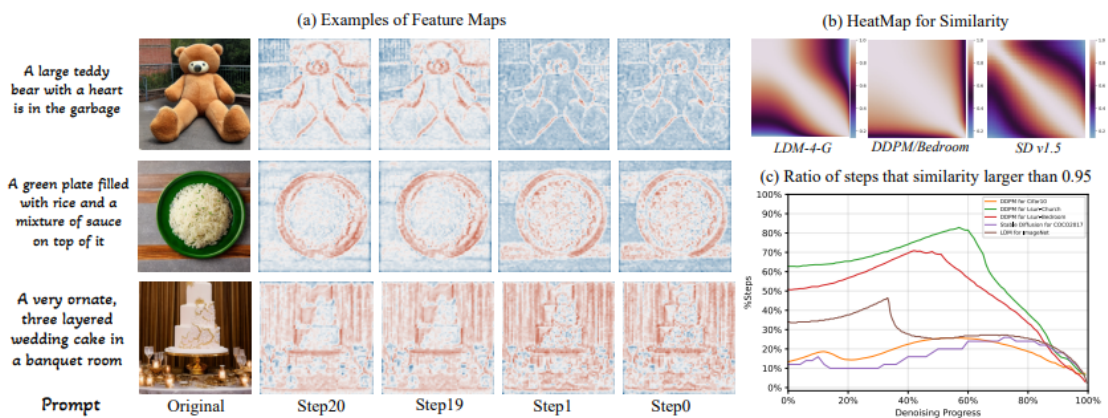


Figure 2. (a) Examples of feature maps in the up-sampling block U_2 in Stable Diffusion. We present a comparison from two adjacently paired steps, emphasizing the invariance inherent in the denoising process. (b) Heatmap of similarity between U_2 's features in all steps on three typical diffusion models. (c) The percentage of steps with a similarity greater than 0.95 to the current step.

- 이러한 특성을 활용하여 denoising 가속화
 - 존재하는 중복 계산을 방지
 - 막대한 계산 비용으로 미미한 변화 뿐이라면 비효율적

3.3. Deep Cache For Diffusion Models

: denoising 과정에서 시간적 중복성을 활용하여 추론 과정 가속화

- (U-net) skip connections : 메인 경로는 전체 네트워크를 거치는 반면, 스킵 경로는 일부의 층만 거치기 때문에 계산 비용이 적음
- 모든 결과 반복 계산 필요 없음

Cacheable Features in denosing.

- x_{t-1} 은 이전 결과 x_t 에 기반하여 조건부 생성 : 이전 출력에서 생성된 고수준 특징을 다음 단계로 가져옴

$$F_{\text{cache}}^t \leftarrow U_{m+1}^t(\cdot)$$

- F 는 주요 특징
- m 번째의 스킵 경로에 필요한 계산만 수행하고, 주요 경로의 계산은 캐시에서 가져 오기

$$\text{Concat}(D_m^{t-1}(\cdot), F_{\text{cache}}^t)$$

- D는 m번째 다운샘플링 블록 출력

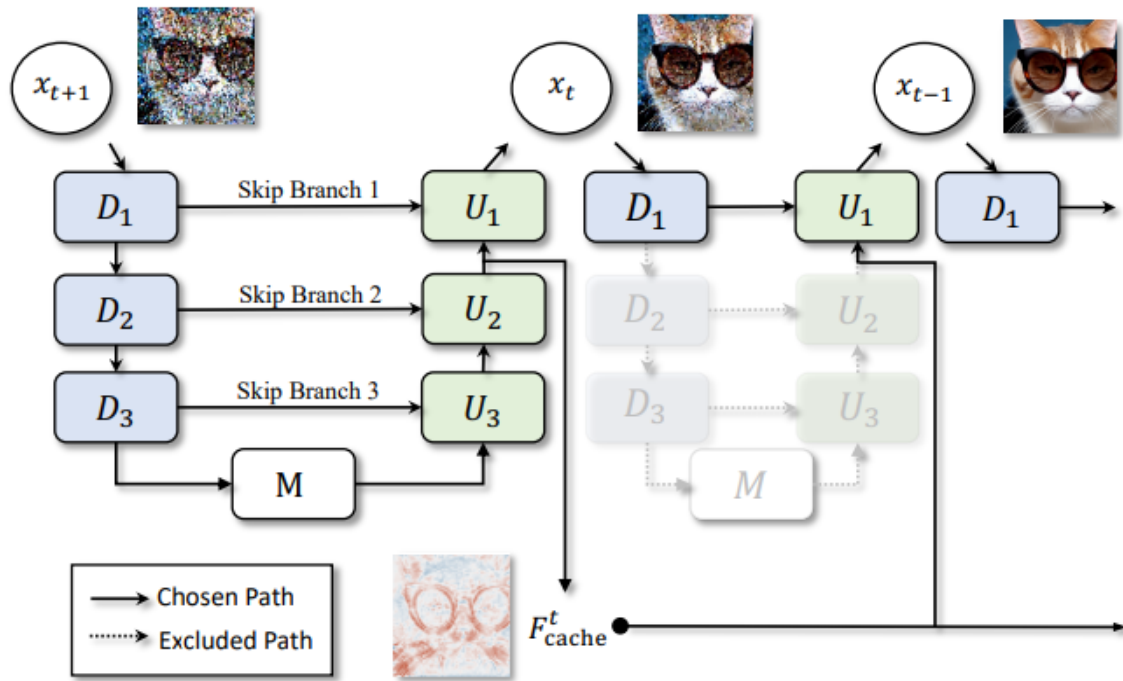


Figure 3. Illustration of DeepCache. At the $t - 1$ step, x_{t-1} is generated by reusing the features cached at the t step, and the blocks D_2, D_3, U_2, U_3 are not executed for more efficient inference.

Extending to 1:N Inference

- 다음 단계가 여러 개인 구조에도 적용 가능
- 한 번 캐시된 피쳐들을 재사용 - 대체하는 N-1 단계

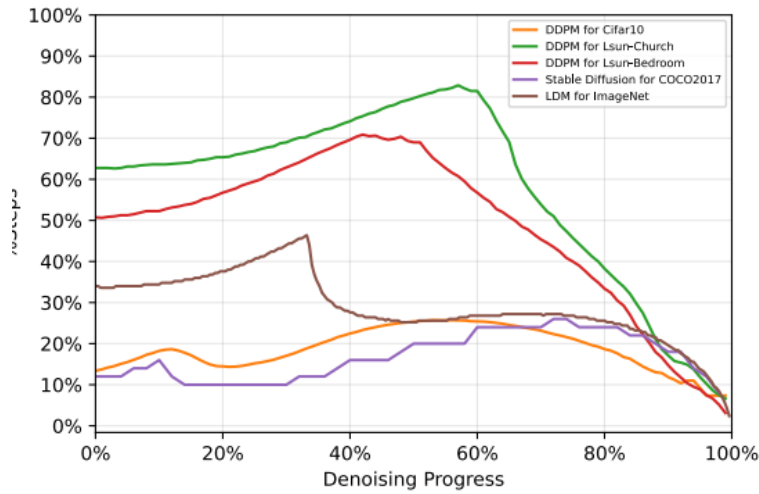
$$\mathcal{I} = \{x \in \mathbb{N} \mid x = iN, 0 \leq i < k\} \quad (6)$$

where $k = \lceil T/N \rceil$ denotes the times for cache updating.

Non-uniform 1:N Inference

- 고수준에서의 특징이 연속적인 N 단계에서 변하지 않는다는 가정
 $\Rightarrow$ 항상 적용 가능하지 않다. 특히 N이 큰 경우에

(c) Ratio of steps that similarity larger than 0.95



피쳐간 유사성이 항상 일정하지 않은 것 확인 가능

non-uniform 에서는 주변 단계들과 상대적으로 작은 유사성을 가지는 단계에서 더 많이 샘플링

$$\mathcal{L} = \left\{ l_i \mid l_i \in \text{linear_space} \left((-c)^{\frac{1}{p}}, (T-c)^{\frac{1}{p}}, k \right) \right\} \quad (7)$$

$$\mathcal{I} = \text{unique_int} \left(\{ i_k \mid i_k = (l_k)^p + c, \text{ where } l_k \in \mathcal{L} \} \right)$$

04 Experiment

4.1. Experimental Settings.

Models, Datasets and Evaluation Metrics

- 사전 학습된 확산 모델의 종류와 관계없이 효과적임을 입증
 - DDPM, LDM, Stable Diffusion
- 샘플링 방법들과 호환성 입증
 - DDPM에는 100단계 DDIM, LDM에는 250단계, Stable Diffusion에는 50단계 PLMS
 - 전체 1000 단계 노이즈 제거 과정 사용 X

- 6가지 모델 평가 데이터셋
 - CIFAR10 , LSUN-Bedroom, LSUN-Churches, ImageNet, MS-COCO 2017, PartiPrompts → 프롬프트로 사용
- 평가 지표
 - FID, sFID, IS, Precision-and-Recall, CLIP Score

Baselines

→ 확산 모델 압축과 → Diff-Pruning 사용

→ 경량화된 확산 모델 재설계

→ Stable Diffusion → BK-SDM 사용

4.2. Complexity Analysis

- MACs 분할 사용으로 추론 과정 가속

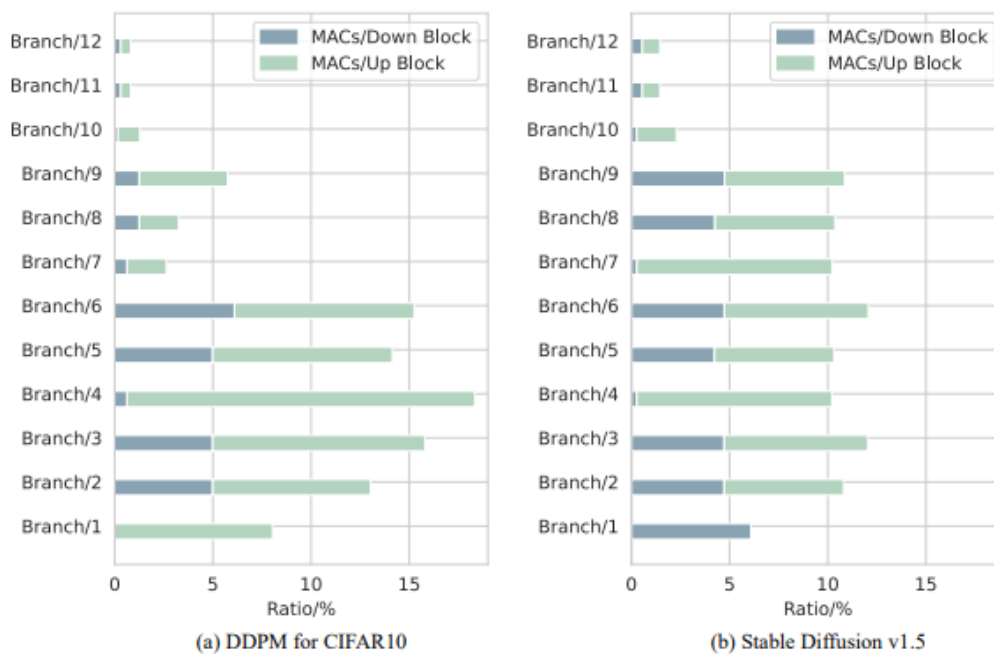


Figure 4. MACs for each skip branch, evaluated on DDPM for CIFAR10 and Stable Diffusion V1.5.

4.3. Comparison with Compression Methods

LDM-4-G for ImageNet.

- ImageNet 에서의 실험

Method	ImageNet 256×256								
	MACs ↓	Throughput ↑	Speed ↑	Retrain	FID ↓	sFID ↓	IS ↑	Precision ↑	Recall ↑
IDDPM [42]	1416.3G	-	-	✗	12.26	5.42	-	70.0	62.0
ADM-G [9]	1186.4G	-	-	✗	4.59	5.25	186.70	82.0	52.0
LDM-4 [47]	99.82G	0.178	1×	✗	3.60	-	247.67	87.0	48.0
LDM-4*	99.82G	0.178	1×	✗	3.37	5.14	204.56	82.71	53.86
Spectral DPM [66]	9.9G	-	-	✓	10.60	-	-	-	-
Diff-Pruning [10]*	52.71G	0.269	1.51×	✓	9.27 _(9.16)	10.59	214.42 _(201.81)	87.87	30.87
Uniform - N=2	52.12G	0.334	1.88×	✗	3.39	5.11	204.09	82.75	54.07
Uniform - N=3	36.48G	0.471	2.65×	✗	3.44	5.11	202.79	82.65	53.81
Uniform - N=5	23.50G	0.733	4.12×	✗	3.59	5.16	200.45	82.36	53.31
Uniform - N=10	13.97G	1.239	6.96×	✗	4.41	5.57	191.11	81.26	51.53
Uniform - N=20	9.39G	1.876	10.54×	✗	8.23	8.08	161.83	75.31	50.57
NonUniform - N=10	13.97G	1.239	6.96×	✗	4.27	5.42	193.11	81.75	51.84
NonUniform - N=20	9.39G	1.876	10.54×	✗	7.11	7.34	167.85	77.44	50.08

Table 1. Class-conditional generation quality on ImageNet using LDM-4-G. The baselines here, as well as our methods, employ 250 DDIM steps. *We reproduce Diff-Pruning to have a comprehensive comparison and the official results are shown in brackets.

가속화 시 약간의 성능 감소

FID, sFID에서 상당한 향상

비균일 1:N 사용 시, 캐시의 간격인 N이 커질수록 향상이 잘 나타남

DDPMs for CIFAR-10 and LSUN.

- CIFAR-10, LSUN-Bedroom, LSUN-Churches에 대한 결과
 - 논문의 방법이 재학습을 요구하는 방법보다 뛰어남 → 재학습 비용이 없음
 - 하드웨어와 친화적인 pruning → 더 나은 가속화

CIFAR-10 32×32					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	6.1G	9.79	1×	-	4.19
DDPM*	6.1G	9.79	1×	-	4.16
Diff-Pruning	3.4G	13.45	1.37×	100k	5.29
Ours - N=2	4.15G	13.73	1.40×	0	4.35
Ours - N=3	3.54G	15.74	1.61×	0	4.70
Ours - N=5	3.01G	18.11	1.85×	0	5.73
Ours - N=10	2.63G	20.26	2.07×	0	9.74

LSUN-Bedroom 256×256					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	248.7G	0.21	1×	-	6.62
DDPM*	248.7G	0.21	1×	-	6.70
Diff-Pruning	138.8G	0.31	1.48×	200k	18.60
Ours - N=2	190.8G	0.27	1.29×	0	6.69
Ours - N=3	172.3G	0.30	1.43×	0	7.20
Ours - N=5	156.0G	0.31	1.48×	0	9.49

LSUN-Churches 256×256					
Method	MACs ↓	Throughput ↑	Speed ↑	Steps ↓	FID ↓
DDPM	248.7G	0.21	1×	-	10.58
DDPM*	248.7G	0.21	1×	-	10.87
Diff-Pruning	138.8G	0.31	1.48×	500k	13.90
Ours - N=2	190.8G	0.27	1.29×	0	11.31
Ours - N=3	172.3G	0.30	1.43×	0	11.75
Ours - N=5	156.0G	0.31	1.48×	0	13.68

Table 2. Results on CIFAR-10, LSUN-Bedroom and LSUN-Churches. All the methods here adopt 100 DDIM steps. * means the reproduced results, which are more comparable with our results since the random seed is the same.

Stable Diffusion

Method	PartiPrompts				COCO2017			
	MACs ↓	Throughput ↑	Speed ↑	CLIP Score ↑	MACs ↓	Throughput ↑	Speed ↑	CLIP Score ↑
PLMS - 50 steps	338.83G	0.230	1.00×	29.51	338.83G	0.237	1.00×	30.30
PLMS - 25 steps	169.42G	0.470	2.04×	29.33	169.42G	0.453	1.91×	29.99
BK-SDM - Base	223.81G	0.343	1.49×	28.88	223.81G	0.344	1.45×	29.47
BK-SDM - Small	217.78G	0.402	1.75×	27.94	217.78G	0.397	1.68×	27.96
BK-SDM - Tiny	205.09G	0.416	1.81×	27.36	205.09G	0.415	1.76×	27.41
Ours	130.45G	0.494	2.15×	29.46	130.45G	0.500	2.11×	30.23

Table 3. Comparison with PLMS and BK-SDM. We utilized prompts in PartiPrompt and COCO2017 validation set to generate images at the resolution of 512. We choose N=5 to achieve a throughput that is comparable to or surpasses that of established baseline methods. Results for other choices of N can be found in Figure 6.

- 노이즈 제거 속도는 더 빠르고
- BK-SDM 세 변형 모델을 모두 능가
 - 기본 확산 모델로 생성된 이미지와 비슷
 - 이미지-텍스트 프롬프트와 더 잘 일치

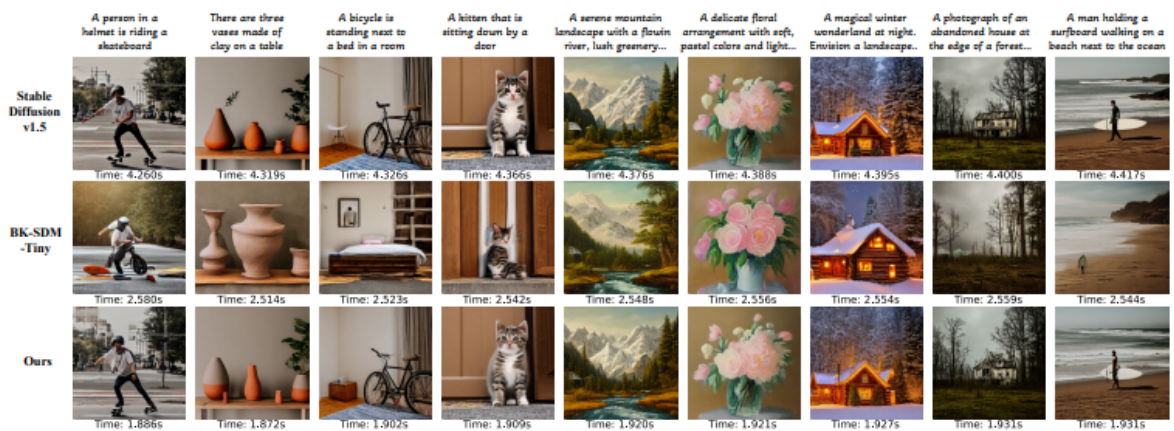


Figure 5. Visualization of the generated images by BK-SDM-Tiny and DeepCache. All the methods adopt the 50-step PLMS. The time here is the duration to generate a single image. Some prompts are omitted from this section for brevity. See Appendix for details.

4.4. Comparison with Fast Sampler.

- 다른 샘플러에 추가적 적용 가능함
- 더 나은 성과

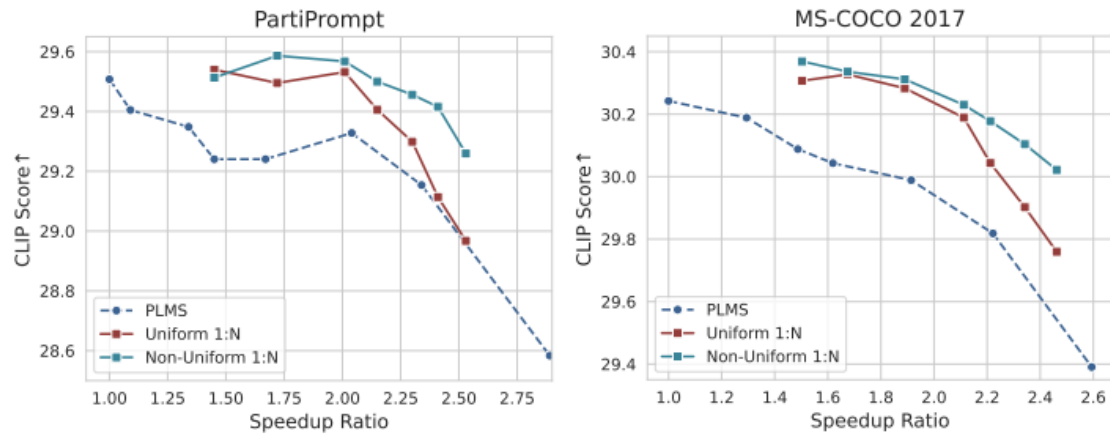


Figure 6. Comparison between PLMS, DeepCache with uniform 1:N and non-uniform 1:N strategies.

4.5. Analysis

- DeepCache : K 단계 위에 얇은 네트워크 추론 추가하고, 계산
 - 캐시된 피쳐는 재학습 없이 U-Net 확산 모델에서 노이즈 제거 과정에 중요 역할
 - 캐시된 피쳐는 DDIN에 비해 좋은 영향 (얇은 추가 계산으로 인함)
- 캐시 간격 N 증가
 - 시간이 단축되는 경향
 - 이미지의 주요 특징이 이전 이미지와 일치
 - 미세 디테일은 변화

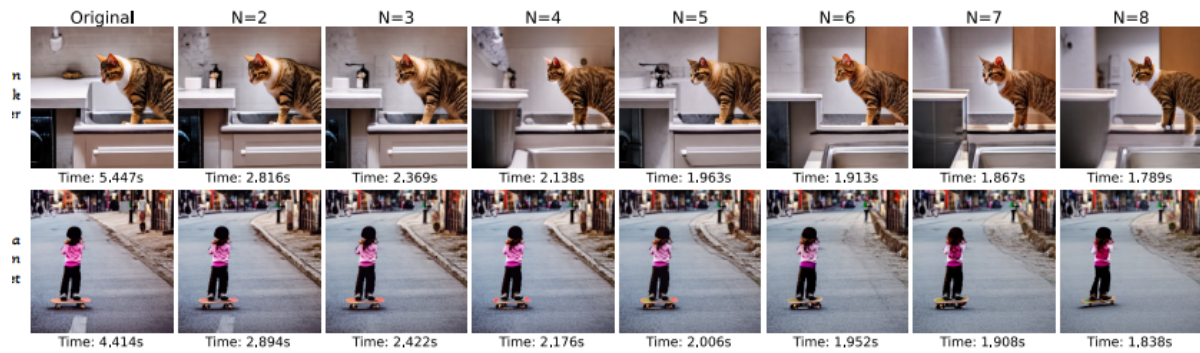


Figure 7. Illustration of the evolution in generated images with increasing caching interval N .

05 Limitations

- 확산 모델의 사전 정의된 구조에 의존 - 스킵 연결 경로에 따라 가속 비율 제한 가능
- 캐싱 간격이 커지면 성능 저하 발생

06 Conclusion

- 확산 모델에서 연속된 단계에서 관찰된 고수준 특징의 유사성 활용 $\Rightarrow$ 중복 계산 완화
- denoising 가속화
- 파라미터 크기 축소에 집중한 다른 경량화 방법을 능가하는 성능
- DDIM, PLMS 등 기존 기술과 동등하거나 더 나은 생성 품질